

1. Kid-level explanation

Imagine you built a little robot to watch over a busy online shop while everyone is asleep at night. One night, the shop starts running very slowly — like a leaky faucet, memory is dripping away inside the computer. Your robot notices immediately. It looks at the code (like reading the shop's recipe book), finds the exact line that is causing the leak, and writes a fix — the same way a doctor writes a prescription. Then it says, "Hey, I found the problem and here is my solution — do you want me to apply it?" You press one button from your phone. The robot sends the fix to the live shop, runs a quick test to make sure nothing broke, and the shop is healthy again. From the moment the leak started to the moment it was fixed: five minutes. No engineers were woken up. No customers noticed anything wrong. That robot is OpsBob. Instead of waiting for a tired human to diagnose a problem at 2 in the morning, OpsBob handles it — and asks for permission before doing anything permanent. It is IBM Bob, but working as the world's most alert and responsible night-shift engineer.

2. Solution overview and IBM tech mapping

Problem & user

Every enterprise running software in production lives in fear of the 2 AM incident. A memory leak spikes. An API endpoint starts throwing 500 errors. A database connection pool exhausts. The on-call SRE gets paged, groggy, opens a laptop, and spends 30–90 minutes just *understanding* the problem before writing a single line of fix. Mean Time To Resolution (MTTR) for production incidents averages 3–4 hours at most Fortune 500 companies. IBM's own Instana product monitors millions of production services for exactly these events — but today, Instana detects and alerts; a human still has to diagnose and fix.

The primary users of OpsBob are: SREs and DevOps engineers who are on-call, engineering managers who own SLA commitments, and platform teams who want to reduce the operational burden of keeping microservice estates healthy. These users are deeply relevant to IBM's enterprise customer base — every IBM Cloud client with more than a handful of services has this pain daily.

High-level solution narrative

OpsBob is a real-time production self-healing platform that wires IBM Instana (via MCP) to IBM Bob's Orchestrator mode. When Instana detects an anomaly — elevated error rate, memory growth, latency spike — it fires an event into

OpsBob's backend. OpsBob immediately opens a Bob Orchestrator session, feeding it the incident context (service name, error signatures, stack traces) and the relevant source code from the affected repository. Bob acts as a senior SRE: it reads the code, identifies the root cause, writes a targeted fix, generates regression tests, and presents a structured "Fix Card" to the engineer on call through a clean dashboard UI. The engineer reviews the diagnosis and the proposed fix in plain English alongside the code diff, then clicks Approve. Bob's BobShell executes a deployment recipe that builds, tests, and pushes the fix to IBM Cloud Code Engine — all within a single governed, auditable session. Every decision Bob makes is logged in the BobShell audit trail, giving compliance teams full traceability.

IBM Bob is not a passenger here. It is the entire reasoning and execution engine: it reads the repository, forms a hypothesis, writes code, writes tests, and orchestrates the deployment. Instana is the sensor. IBM Cloud is the runtime. Bob is the surgeon.

IBM technology mapping

IBM Bob

- Orchestrator Mode: The top-level agent that receives the incident brief and coordinates the full diagnosis-to-deploy loop. This is Bob's highest-sophistication mode and the one judges most want to see demonstrated.
- Ask Mode: Initial step — Bob reads the affected service codebase and incident stack trace, builds context without making changes.
- Plan Mode: Bob produces a structured root-cause analysis and a proposed fix strategy with explicit reasoning. This is shown live on the dashboard as a "Bob's Diagnosis Card."
- Code Mode: Bob writes the actual code fix (e.g., clearing a HashMap that was never being evicted, fixing a connection pool leak, adding a null check before an NPE-prone path).
- Code Mode (Test Generation): Bob generates regression tests covering the fixed code path.
- BobShell: Executes the build, runs the test suite, packages the container, and pushes to IBM Cloud Code Engine. The BobShell audit log is shown as a real-time activity feed in the dashboard.

Example Bob prompts used internally:

- [Ask] Analyze this Node.js service codebase. Focus on memory management patterns, object lifecycle, and any collections that grow without bound.
- [Plan] Given this Instana alert (memory growth +200MB over 10 min, service: payments-api), identify the most likely root cause and propose a minimal fix.
- [Code] Implement the fix for the identified memory leak. Write a targeted unit test that would catch this regression.
- [BobShell] Run the test suite, build the Docker image, push to IBM Cloud Code Engine endpoint payments-api-fixed.

IBM Instana (via MCP) OpsBob integrates with Instana through an MCP server that exposes three tools: `get_active_incidents`, `get_service_metrics(service, window)`, and `get_stack_traces(incident_id)`. This MCP server is the *trigger layer* of the system — it is what makes OpsBob event-driven rather than batch. In the demo, Instana's alert fires the entire autonomous pipeline.

IBM Granite (via Bob's multi-model routing) Bob's multi-model orchestration routes the fast, structured tasks (test generation, build verification) to lighter Granite SLMs for cost efficiency, while complex root-cause reasoning goes to more capable models. This directly demonstrates IBM's core differentiator — cost-optimized model routing — and can be called out explicitly in the demo narration.

IBM Cloud Code Engine The target deployment runtime for the fixed microservice. The BobShell deployment recipe pushes a new container revision to Code Engine in approximately 30 seconds. The dashboard shows the live URL flip from "degraded" to "healthy" with a latency graph from Instana confirming recovery. This is the visual payoff moment.

BobShell audit trail Every Bob action — every reasoning step, every code change, every test result, every deployment command — is logged in a structured JSON audit trail by BobShell. The OpsBob dashboard renders this as a real-time "Incident Timeline" that a compliance team could use to answer "what did the AI do and why?" after any automated fix. This directly addresses IBM's stated governance narrative.

3. Architecture and build plan (before deadline)

Architecture

The system has five components:

1. Incident Monitor (Instana MCP Server) A lightweight Node.js MCP server that wraps the Instana REST API. It exposes three MCP tools to Bob: `get_active_incidents`, `get_service_metrics`, and `get_stack_traces`. For the demo, a separate script injects a real memory leak into the demo Node.js microservice and triggers a mock Instana webhook to simulate a production alert. This mock is transparent and stated openly in the README — it's a demo scaffold, not a fake.
2. Demo Microservice (the patient) A small Node.js Express service (two endpoints, deliberately simple) with an injected bug — specifically, a Map that accumulates session objects and is never cleared, causing measurable memory growth over 2–3 minutes of traffic. This is the real target Bob will diagnose and fix.
3. OpsBob Backend (Python FastAPI) The orchestration layer. Receives the Instana webhook, fetches incident details and stack traces via the Instana MCP server, constructs the context payload for Bob (incident summary + relevant source files), calls Bob's API in Orchestrator mode, streams the response back to the frontend via Server-Sent Events (SSE), and on approval triggers BobShell to execute the deployment recipe.
4. OpsBob Dashboard (React) Three-panel layout:
 - Left panel: Incident Feed — shows active Instana alerts with severity, service name, and incident timestamp
 - Center panel: Bob's Diagnosis Card — streams Bob's root-cause reasoning, proposed fix, and code diff with syntax highlighting
 - Right panel: Fix Actions — "Approve Fix" and "Reject / Escalate" buttons, plus a deployment status indicator

The entire UI is built with IBM Carbon Design System components. Any DevOps engineer who has ever used PagerDuty or OpsGenie will immediately understand this screen.

5. IBM Cloud Code Engine (deployment target) The demo microservice runs on Code Engine as an initial (buggy) revision. On approval, BobShell triggers a new revision deployment using the IBM Cloud CLI. The Instana metrics panel on the dashboard shows the memory graph flatten and latency normalize after the new revision goes live.

Data flow

The system works as follows:

1. A load-generation script hits the demo microservice, causing the memory leak to manifest within 2–3 minutes.
2. The mock Instana webhook fires to the OpsBob backend with { service: "payments-api", anomaly: "memory_growth", severity: "high" }.
3. The backend calls the Instana MCP server's get_stack_traces tool, retrieving a realistic stack trace pointing to the leaking function.
4. The backend assembles a context payload: incident summary + stack trace + the three relevant source files from the repo.
5. The backend calls Bob API in Orchestrator mode with this context and a system prompt defining Bob's role as a production SRE agent.
6. Bob's response streams back through SSE: first the Ask-mode summary, then the Plan-mode diagnosis, then the Code-mode fix diff.
7. The dashboard renders each phase in real time as it arrives.
8. The on-call engineer reads the diagnosis, reviews the diff, and clicks "Approve Fix."
9. The backend triggers BobShell: run tests → build container → push to Code Engine.
10. The BobShell output streams to the "Incident Timeline" panel on the dashboard.
11. The new Code Engine revision goes live. The Instana metrics (polled every 10 seconds) show memory stabilizing. The incident card closes as "Resolved by Bob."

Build plan

Phase 0 — Setup (Hours 1–3)

- Clone IBM Bob SDK documentation, register for API access, confirm Bob API calls work locally
- Set up IBM Cloud account, create Code Engine project, deploy the demo microservice (buggy version)
- Scaffold FastAPI backend (Python) with a /webhook endpoint and a /stream-bob SSE endpoint

- Scaffold React frontend with the three-panel Carbon layout (no logic yet, just structure)
- Write the memory-leak microservice (Node.js, 60 lines of code)
- Write the load generator script (30 lines)
- *Bob prompts to use:* [Ask] Scaffold a Python FastAPI application with a POST /webhook endpoint and a GET /stream SSE endpoint for real-time streaming

Phase 1 — Core happy path (Hours 3–14)

- Build the Instana MCP server wrapper (mock/real) — two Node.js files, roughly 80 lines
- Wire the backend to call Bob Orchestrator API with incident context and source files
- Stream Bob's multi-phase response (Ask → Plan → Code) back to the frontend
- Render streaming text in the center panel of the dashboard
- Wire the "Approve" button to call the BobShell deployment recipe
- Test the full loop end-to-end: inject bug → webhook → Bob diagnoses → approve → Code Engine updates
- *Bob prompts to use:* [Code] Build a Node.js MCP server that wraps these two Instana REST API endpoints: GET /api/v1/events and GET /api/v1/infrastructure-monitoring/traces. Expose them as MCP tools get_active_incidents and get_stack_traces.

Phase 2 — Demo polish (Hours 14–24)

- Add the Incident Feed left panel with severity color coding (red/yellow/green)
- Add the BobShell audit trail as a scrolling log in the right panel below the action buttons
- Add a memory metric sparkline graph (pull from Instana or mock a realistic curve)
- Add the "Resolved by Bob" animation when the new Code Engine revision is confirmed healthy

- Write the demo script — the exact 3-minute narration of the 2:14 AM scenario
- Record a test run-through, check for any rough edges
- *Bob prompts to use:* [Code] Build a React component using IBM Carbon Design System that shows a real-time streaming text panel with three sections: "Diagnosis," "Proposed Fix" (with syntax-highlighted diff), and "Test Results"

Phase 3 — Stretch features (Hours 24–36, only if Phase 2 is clean)

- Add a second demo scenario: a null pointer exception bug (different failure mode, same pipeline)
- Add IBM Secrets Manager for credential injection into the fixed container
- Add a simple "Incident History" view showing past Bob-resolved incidents
- Add a "Cost Card" showing how much compute Bob spent on this resolution (Bobalytics-style)
- *Bob prompts to use:* [Orchestrator] Given this list of past incidents and their Bob resolution steps, generate a concise "Lessons Learned" summary that could seed a runbook

4. How we hit 5/5 on all criteria

Completeness & feasibility — 5/5

The demo path is a single, unbroken loop: inject bug → Instana alert fires → Bob diagnoses live on screen → engineer approves → Code Engine deploys fixed revision → metrics normalize. There are no "and then imagine this happens" moments. The demo microservice is a real Node.js service running on IBM Cloud, not a mock output. The Bob API calls are real. The Code Engine deployment is real. Every IBM technology requirement is satisfied with a concrete, visible component: Bob (Orchestrator/Ask/Plan/Code/BobShell), Instana MCP (integration tool), IBM Cloud Code Engine (deployment), Granite models (via Bob's internal routing). The two-person DRAGORITHM team can build this in 36–40 hours because the demo microservice is intentionally tiny — two endpoints, one injected bug — and every phase of the build plan has clear, bounded tasks with no unknown unknowns.

Creativity & innovation — 5/5

Every other submission will show Bob inside a developer's IDE, helping someone write or modernize code. OpsBob removes Bob from the IDE entirely and deploys it as an autonomous production guardian. This is conceptually a category leap — not "AI-assisted coding" but "AI-driven production resilience." IBM has published extensively on the Blue Pearl and Maximo modernization wins, but has not yet demonstrated Bob as an autonomous SRE operating on live production incidents. OpsBob is the next evolution IBM themselves haven't fully published yet. The Instana MCP integration is particularly sharp: it uses IBM's own observability product as the sensor layer, creating a genuinely IBM-native end-to-end story. The recursive element — Bob monitors the code that Bob can also fix — is a narrative IBM executives will want to retell at customer briefings.

Design & usability — 5/5

The three-panel dashboard maps perfectly to the mental model of every on-call engineer: "What is broken? What does Bob think? What do I do?" The layout requires zero training. The visual payoff — watching the memory graph flatten after the approved fix goes live — converts the entire technical process into one emotionally satisfying moment that any non-developer in the judging room will understand. IBM Carbon Design System ensures the UI looks enterprise-ready rather than hackathon-built. The demo narrative ("2:14 AM") gives judges a human story to anchor the technology to, which is the most reliable way to make a demo stick in memory. The approval button in the center of the screen is the embodiment of IBM's governance narrative: "AI acts, humans decide."

Effectiveness & efficiency — 5/5

OpsBob targets MTTR — Mean Time To Resolution — the single most important operational KPI for every SRE team on earth. Industry average MTTR for production incidents is 3–4 hours. OpsBob's demo scenario resolves in under 5 minutes, including Bob's diagnostic time. That is a 95%+ reduction in a metric that directly maps to SLA breach risk, customer churn, and engineering burnout. Every Fortune 500 IBM enterprise customer has this problem on at least a weekly basis. IBM's Instana product already sits in many of these customers' production environments — OpsBob is an immediate, credible add-on to an existing IBM product relationship. The scaling story is equally strong: the same architecture works for any language, any cloud-native service, any Instana-monitored environment. IBM could present this as a Bob Premium Package for SRE teams.

5. Uniqueness and anti-collision check

What most teams will build

Based on the hackathon theme and the 14 ideas in your own list, the most common submissions will likely cluster into these five patterns:

1. "Bob modernizes legacy Java/COBOL" — easily 30–40% of all teams. Variants: CodeRevive, Flightdeck, CodeBridge, SGF, ModernBridge, LegacyBridge Gov all essentially land here.
2. "Bob generates MCP servers from API specs or natural language" — easily another 20–25% of teams. LegacyBridge, AgentForge, FlowForge, ContextForge are all this pattern.
3. "Bob as full SDLC automation pipeline" — SpecBridge, DevFlow Studio land here. Maybe 10% of teams.
4. "Bob as enterprise chatbot for IT/HR/procurement" — generic agent chat UI. Maybe 10% of teams.
5. "Bob + watsonx for a specific industry vertical" — healthcare, finance, government automation. Maybe 10% of teams.

Why OpsBob is meaningfully different from all five patterns

1. Bob leaves the IDE entirely. Every pattern above is developer-tooling. OpsBob is production-operations tooling. The user is an SRE at 2 AM, not a developer at their desk.
2. The trigger is an external system event, not a human prompt. OpsBob is event-driven: Instana fires, Bob reacts. No other idea in the list has this architecture.
3. The domain is incident response, not code creation. While every other team is in the "build new things" space, OpsBob is in the "heal broken things" space. IBM doesn't have a published reference architecture for this yet.
4. The KPI is operational, not developmental. MTTR is something a CIO can put in a board presentation. "Lines of code modernized per hour" is not.
5. It creates pull for IBM's existing product. IBM Instana is already installed in enterprise production environments. OpsBob doesn't ask customers to adopt a new system — it augments one they already paid for. Judges who are IBM product leaders will immediately see the commercial path.

Differentiation tweaks if needed

If you want to sharpen uniqueness further without adding scope, consider two optional angles:

- Narrow the vertical to financial services: frame the demo as a payment-processing microservice. Financial SREs have the highest-stakes on-call obligations, the strictest audit requirements, and the most budget for tools that reduce MTTR. IBM has deep FS relationships. This framing makes the business case concrete without changing a line of code.
 - Add a "Bob learns from history" angle: after each resolved incident, BobShell writes a summary to a simple Cloudant document. On the next incident of the same type, Bob's context prompt includes the previous resolution. This creates a lightweight institutional memory feature that reinforces IBM's governance narrative and adds one compelling screenshot to the demo with very little additional build time.
-

6. Why this idea can win

OpsBob is the only idea in your list that takes IBM Bob out of the developer workspace and deploys it as a production-time autonomous agent — detecting, diagnosing, fixing, and deploying in a fully auditable loop while a human maintains final approval authority. It scores 5/5 on all four criteria because the demo path is a single, unbroken real system (real bug, real Bob analysis, real Code Engine deployment, real metric recovery), the concept is genuinely novel in a field where 60% of submissions will be legacy modernization or MCP generation variants, the three-panel dashboard is instantly legible to any DevOps engineer or IBM executive in the room, and the MTTR metric gives judges a number they can repeat in a slide deck. Most importantly, it tells IBM's most persuasive sales story — not "Bob helps developers write code faster," but "Bob protects your production revenue while your team sleeps" — and that is the story IBM's enterprise customers will pay for.